MASTER PROFESSIONNEL FAVD

Visualisation des Données Massives

Rapport de Mini-Projet

Conception et Déploiement d'un Data Warehouse Retail

Multi-Environnements

Étudiantes	Safa Jlassi & Ines Boujamaa
Formation	Master Professionnel FAVD
Matière	Visualisation des Données Massives
Technologies	MySQL Cluster NDB · Docker · Python · Streamlit
Année	2025 – 2026

1. Introduction

1.1 Contexte

Les entreprises du secteur Retail exploitent des volumes croissants de données issues de multiples sources : systèmes de caisse (POS), ERP, plateformes e-commerce et programmes de fidélité. La dispersion de ces données dans des systèmes hétérogènes rend difficile toute analyse décisionnelle cohérente et rapide.

Dans ce contexte, la mise en place d'un Data Warehouse (DWH) structuré et scalable est devenue indispensable pour centraliser, consolider et valoriser ces données à des fins analytiques.

1.2 Objectif du Projet

Ce mini-projet nous place dans le rôle d'une équipe Data Engineering chargée de concevoir, déployer et documenter une architecture décisionnelle complète pour une chaîne de distribution retail tunisienne fictive : RetailTN.

Les objectifs principaux sont :

- Concevoir un modèle multidimensionnel en schéma en étoile adapté à l'analyse des ventes
- Déployer un cluster MySQL NDB distribué et haute disponibilité
- Implémenter un pipeline ETL Python automatisé avec chargements initial et incrémental
- Mettre en place quatre environnements Docker isolés : DEV, TEST, PROD, DAC
- Visualiser les indicateurs clés de performance (KPIs) dans un dashboard Streamlit interactif

1.3 Présentation de RetailTN

RetailTN est une chaîne de distribution fictive opérant sur l'ensemble du territoire tunisien avec :

- 50 magasins répartis sur les 24 gouvernorats
- 200 produits dans 5 catégories : Électronique, Vêtements, Alimentation, Maison, Sport
- 5 000 clients segmentés en Gold, Silver et Bronze
- 20 promotions sur la période 2022-2024
- 50 000 transactions de vente sur 3 ans (2022-2024)

2. Architecture Proposée

2.1 Vue Globale

L'architecture du projet repose sur quatre couches distinctes qui assurent la séparation des responsabilités et la modularité du système :

Couche	Composant	Rôle	Technologie
1. Source	Faker Python	Génération de données simulées réalistes	Python + Faker fr_TN
2. ETL	Pipeline Python	Extraction, transformation, chargement	Python + pandas + SQLAlchemy
3. DWH	MySQL Cluster NDB	Stockage distribué schéma en étoile	MySQL 8.0 NDB Engine
4. Présentation	Dashboard Streamlit	Visualisation des 5 KPIs	Python + Streamlit

2.2 Architecture Multi-Environnements

Conformément aux spécifications de l'énoncé, quatre environnements Docker isolés ont été configurés, chacun avec son propre réseau, ses volumes persistants et ses variables d'environnement :

Environnement	Dossier	Port MySQL	Réseau Docker	Rôle
DEV	dev/	3307	dwh_dev_net	Développement et debug
TEST	test/	3308	dwh_test_net	Validation et tests automatisés
PROD	prod/	3309	dwh_prod_net	Production avec cluster NDB complet
DAC	dac/	3310	dwh_dac_net	Disaster Recovery & Analytics

2.3 Architecture MySQL Cluster NDB

MySQL Cluster NDB est une architecture de base de données distribuée qui stocke les données en mémoire sur plusieurs nœuds. Cette architecture garantit la haute disponibilité et la tolérance aux pannes.

Le cluster déployé dans l'environnement DEV comprend quatre nœuds :

Nœud	Image Docker	Rôle	Port
Management Node (ndb_mgmd)	mysql/mysql-cluster:8.0	Coordination et configuration du cluster	1186
Data Node 1 (ndbd1)	mysql/mysql-cluster:8.0	Stockage distribué des	-

		données NDB	
Data Node 2 (ndbd2)	mysql/mysql-cluster:8.0	Réplication et haute disponibilité	-
SQL Node (mysqld)	mysql/mysql-cluster:8.0	Interface SQL standard (port exposé)	3307

Toutes les tables du DWH utilisent ENGINE=NDB, ce qui force MySQL à stocker les données sur les Data Nodes distribués plutôt que sur un seul serveur.

3. Modélisation

3.1 Choix du Modèle : Schéma en Étoile

Nous avons opté pour un schéma en étoile (Star Schema) pour les raisons suivantes :

- Simplicité des jointures : toutes les dimensions pointent directement vers la table de faits
- Performance optimale pour les requêtes OLAP analytiques
- Facilité de compréhension pour les utilisateurs métier
- Compatibilité native avec les outils de Business Intelligence

3.2 Granularité

La granularité choisie est : 1 ligne = 1 transaction de vente, soit 1 produit acheté par 1 client dans 1 magasin à 1 date précise, avec éventuellement 1 promotion appliquée.

Ce niveau de granularité permet d'agréger les données à n'importe quel niveau d'analyse : journalier, mensuel, annuel, par produit, par région, par segment client...

3.3 Table de Faits : fact_sales

La table de faits centralise toutes les mesures quantitatives de chaque transaction :

Colonne	Type SQL	Rôle	Nature
sale_id	BIGINT AUTO_INCREMENT	Identifiant unique	**Clé PK**
date_id	INT NOT NULL	Référence dim_date	Clé FK
product_id	INT NOT NULL	Référence dim_product	Clé FK
customer_id	INT NOT NULL	Référence dim_customer	Clé FK
store_id	INT NOT NULL	Référence dim_store	Clé FK
promo_id	INT	Référence dim_promotion	Clé FK opt.
quantity	INT NOT NULL	Quantité vendue	Mesure
unit_price	DECIMAL(10,2)	Prix unitaire HT	Mesure
total_ht	DECIMAL(12,2)	Montant HT	Mesure
discount_pct	DECIMAL(5,2)	Taux de remise (%)	Mesure
total_ttc	DECIMAL(12,2)	Montant TTC (TVA 19%)	Mesure
cost_price	DECIMAL(10,2)	Coût d'achat	Mesure
gross_margin	DECIMAL(12,2)	Marge brute = total_ht - coût	Mesure

3.4 Tables de Dimensions

Dimension	Nb colonnes	Colonnes principales	Nb lignes
dim_date	10	date_id, full_date, year, quarter, month, month_name, week_iso, day_name, is_weekend, is_holiday	1 096
dim_product	7	product_id, sku, product_name, category, sub_category, brand, list_price	200
dim_store	8	store_id, store_code, store_name, city, governorate, region, store_type, surface_sqm	50
dim_customer	8	customer_id, customer_code, full_name, city, governorate, age_group, segment, loyalty_points	5 000
dim_promotion	6	promo_id, promo_name, promo_type, discount_rate, start_date, end_date	20

3.5 Les 5 KPIs Retail

KPI	Question Métier	Table(s) impliquée(s)	Page Dashboard
KPI A	Quel magasin génère le plus de CA ?	fact_sales + dim_store	CA par Magasin
KPI B	Comment évoluent les ventes chaque mois ?	fact_sales + dim_date	Evolution Ventes
KPI C	Quels sont les 10 produits les plus vendus ?	fact_sales + dim_product	Top Produits
KPI D	Les promotions font-elles vendre plus ?	fact_sales + dim_promotion	Promotions
KPI E	Quel segment client dépense le plus ?	fact_sales + dim_customer	Panier Moyen

4. Implémentation

4.1 Génération des Données (generate_data.py)

Les données ont été générées avec la bibliothèque Python Faker (locale fr_TN) pour simuler des données réalistes tunisiennes. Le script génère automatiquement les 6 tables en CSV dans data/raw/ :

Table générée	Nb lignes	Méthode de génération
dim_date	1 096	Boucle sur toutes les dates 2022-01-01 → 2024-12-31
dim_product	200	Faker.word() + catégories et marques aléatoires
dim_store	50	50 magasins sur les 24 gouvernorats tunisiens
dim_customer	5 000	Faker.name() + gouvernorats + segments Gold/Silver/Bronze
dim_promotion	20	20 promotions avec dates et taux de remise aléatoires
fact_sales	50 000	50 000 transactions avec calculs auto : TVA 19%, marge brute

4.2 Pipeline ETL

Le pipeline ETL est organisé en 5 fichiers Python dans data/etl/, chacun avec une responsabilité unique :

Fichier	Phase ETL	Rôle principal
extract.py	Extract	Lit les CSV depuis data/raw/, valide les colonnes et les types
transform.py	Transform	Supprime les doublons, remplace les nulls, normalise les textes, recalcule les marges
load_initial.py	Load	Charge TOUTES les données dans MySQL (première exécution)
load_incremental.py	Load	Charge uniquement les NOUVELLES données (basé sur date_id)
pipeline.py	Orchestration	Coordonne les 3 phases dans l'ordre, gère les logs et les erreurs

Règles de transformation appliquées dans transform.py

- Suppression des doublons : drop_duplicates() sur les colonnes clés métier (ex: sku pour les produits)
- Remplacement des valeurs nulles : fillna() avec des valeurs par défaut (segment='Bronze', discount_pct=0)
- Normalisation des textes : strip().title() pour uniformiser les noms et villes
- Validation des données : suppression des prix négatifs et des remises > 100%

- Recalcul de la marge brute : gross_margin = total_ht - quantity × cost_price pour garantir la cohérence

4.3 Organisation du Repository GitHub

La structure du repository respecte exactement les spécifications de l'énoncé :

Dossier / Fichier	Contenu	Nb fichiers
README.md	Présentation, architecture, installation, résultats	1
architecture/	4 diagrammes Mermaid : star schema, ETL sequence, classes Python, architecture globale	4
dev/ + test/ + prod/ + dac/	docker-compose.yml pour chaque environnement	4
sql/	Scripts SQL : création tables, contraintes, vues KPI, requêtes exemples	4
data/	generate_data.py + 5 fichiers ETL + 3 fichiers de tests	9
analysis/	app.py + 5 pages KPI + utils (db.py, charts.py)	9
.gitignore	Exclusion des fichiers temporaires (__pycache__, .env, data/raw/)	1

5. Tests Automatisés

Conformément aux bonnes pratiques du Data Engineering, un suite de tests automatisés a été mise en place avec pytest. 24 tests ont été écrits et tous passent avec succès.

5.1 Organisation des Tests

Fichier de test	Nb tests	Ce qui est testé
test_extract.py	6	Lecture des CSV, présence des colonnes, absence de nulls sur les FK, gestion des fichiers manquants
test_transform.py	11	Suppression des doublons, remplacement des nulls, normalisation des textes, calcul des marges, validation des remises
test_load.py	7	Création des loaders, ordre de chargement (dimensions avant faits), filtrage incrémental, gestion des DataFrames vides

5.2 Résultats

Métrique	Valeur
Tests écrits	24
Tests passés	**24 ✓**
Tests échoués	**0**
Commande d'exécution	python -m pytest data/tests/ -v

5.3 Choix Technique : pytest-mock

Pour les tests de chargement (test_load.py), nous avons utilisé pytest-mock avec MagicMock pour simuler la connexion MySQL. Cette approche permet de tester la logique du code sans avoir besoin d'une vraie base de données, rendant les tests rapides, isolés et reproductibles dans n'importe quel environnement.

6. Résultats Analytiques

6.1 Données Chargées

Table	Nb lignes chargées	Statut
dim_date	1 096	✓ Chargée
dim_product	200	✓ Chargée
dim_store	50	✓ Chargée
dim_customer	5 000	✓ Chargée
dim_promotion	20	✓ Chargée
fact_sales	50 000	✓ Chargée
TOTAL	**56 366**	**✓ Succès**

6.2 Requêtes SQL des 5 KPIs

KPI A — Chiffre d'Affaires par Magasin

Cette requête agrège les ventes par magasin et calcule le CA total, le nombre de transactions et le panier moyen. Elle permet d'identifier les magasins les plus performants et les régions à fort potentiel.

KPI B — Evolution Mensuelle des Ventes

Cette requête groupe les ventes par année et mois pour visualiser la tendance temporelle du chiffre d'affaires. Elle permet de détecter les saisonnalités et les tendances de croissance.

KPI C — Top 10 Produits

Cette requête classe les produits par chiffre d'affaires décroissant. Elle permet d'identifier les produits stars qui génèrent le plus de revenus et orienter les décisions d'achat.

KPI D — Impact des Promotions

Cette requête analyse l'effet des promotions sur le volume de ventes et le CA. Elle joint fact_sales avec dim_promotion pour calculer le CA généré lors des périodes promotionnelles.

KPI E — Panier Moyen par Segment Client

Cette requête calcule le panier moyen par segment (Gold, Silver, Bronze) et par tranche d'âge. Elle permet de personnaliser les offres commerciales selon le profil client.

6.3 Dashboard Streamlit

Le dashboard interactif est accessible sur http://localhost:8501 et comprend 5 pages correspondant aux 5 KPIs. Chaque page offre :

- Des filtres interactifs (région, catégorie, année, segment...)
- Des métriques clés affichées en haut de page
- Des graphiques dynamiques (bar chart, line chart)
- Un tableau détaillé avec toutes les données
- Un cache de 5 minutes pour optimiser les performances (st.cache_data)

7. Discussion

7.1 Points Forts du Projet

- Architecture complète et fonctionnelle : les 4 couches (source, ETL, DWH, visualisation) sont opérationnelles
- Code modulaire et maintenable : chaque fichier a une responsabilité unique (principe SOLID)
- Tests automatisés : 24/24 tests passés garantissent la qualité du code
- Documentation complète : README, 4 diagrammes Mermaid, code commenté
- Multi-environnements : séparation claire DEV/TEST/PROD/DAC avec Docker Compose
- Données réalistes : Faker fr_TN produit des données contextualisées pour la Tunisie

7.2 Difficultés Rencontrées

- Configuration du cluster MySQL NDB : la syntaxe du fichier mysql-cluster.cnf est très stricte. Windows ajoute des caractères invisibles lors de la sauvegarde qui causent des erreurs de parsing. Solution : réécriture du fichier via Add-Content dans PowerShell.
- Ordre de démarrage des nœuds : le Management Node doit démarrer avant les Data Nodes. Docker Compose gère cela avec depends_on, mais les healthchecks NDB nécessitent un temps d'initialisation.
- Compatibilité ENGINE=NDB : certaines contraintes SQL (FOREIGN KEY) ne sont pas supportées par NDB Engine et ont été remplacées par des index.

7.3 Choix Techniques Justifiés

Choix	Alternative	Justification
Streamlit	Power BI / Grafana	Open source, entièrement en Python, connexion MySQL native, recommandé par le prof
SQLAlchemy	mysql-connector seul	Abstraction SQL plus robuste, gestion des transactions, compatibilité multi-SGBD
pandas	PySpark	Volume de données (50 000 lignes) ne justifie pas Spark, pandas est plus simple et rapide
pytest-mock	Base de données de test	Tests plus rapides, isolés, sans dépendance externe
Faker fr_TN	Dataset Kaggle réel	Données contextualisées tunisiennes, contrôle total du volume et de la distribution

8. Conclusion

Ce projet nous a permis de mettre en pratique l'ensemble du cycle de vie d'un projet Data Warehouse, depuis la modélisation conceptuelle jusqu'à la visualisation des données en passant par l'implémentation d'un cluster distribué.

Les compétences développées couvrent plusieurs domaines complémentaires :

- Modélisation multidimensionnelle : conception d'un schéma en étoile avec justification de la granularité
- Data Engineering : pipeline ETL Python robuste avec gestion des erreurs, logging et idempotence
- Administration de bases de données distribuées : déploiement d'un cluster MySQL NDB avec 4 nœuds
- DevOps : conteneurisation avec Docker Compose et séparation multi-environnements
- Développement d'applications : dashboard Streamlit interactif avec 5 KPIs
- Qualité logicielle : suite de 24 tests automatisés avec pytest

Le projet est entièrement fonctionnel, documenté et disponible sur GitHub. Les 50 000 transactions simulées permettent de tester toutes les fonctionnalités analytiques du système.

En perspective, plusieurs améliorations pourraient enrichir ce projet : l'ajout d'un monitoring du cluster avec Prometheus et Grafana, la mise en place d'un pipeline CI/CD avec GitHub Actions, ou l'intégration d'un modèle prédictif de prévision des ventes basé sur les tendances historiques.